

# **Pipelined Reliable Transfer Protocol**

*Connection-oriented reliability, flow control, congestion control, and recovery over UDP*

**By:**

**Jasleen Kaur (301578232)**

**&**

**Rodrigo Fidel (301591985)**

**CMPT 371 | Mini Project | Option (2)**

## Table of Contents

|                                                                      |              |
|----------------------------------------------------------------------|--------------|
| <b>1. Introduction</b>                                               | <b>3</b>     |
| <b>2. Design and Implementation</b>                                  | <b>3-7</b>   |
| 2.1 Why UDP                                                          | 3            |
| 2.2 Custom segment format                                            | 3-4          |
| 2.3 Connection establishment                                         | 4            |
| 2.4 Pipelining and flow control                                      | 4-5          |
| 2.5 Congestion Control – Tahoe-inspired                              | 5            |
| 2.6 Adaptive Retransmission timeout                                  | 6            |
| 2.7 Connection termination                                           | 6            |
| 2.8 Design summary                                                   | 6-7          |
| <b>3. Implementation Specifics</b>                                   | <b>7-9</b>   |
| 3.1 Source Structure                                                 | 7            |
| 3.2 Sender algorithm                                                 | 7            |
| 3.3 Receiver algorithm                                               | 8            |
| 3.4 Controlled loss and corruption                                   | 8            |
| 3.5 Recovery behaviour after timeout                                 | 8-9          |
| <b>4. Test Methodology and Reproducibility</b>                       | <b>9-11</b>  |
| 4.1 Environmental and default parameters                             | 9            |
| 4.2 Run procedures                                                   | 10           |
| 4.3 Test matrix                                                      | 11           |
| 4.4 End-to-end correctness check                                     | 11-12        |
| <b>5. Traffic Analysis – Wireshark Evidence</b>                      | <b>12-17</b> |
| 5.1 Three-way handshake                                              | 12-13        |
| 5.2 Flow-control evidence                                            | 14           |
| 5.3 Corruption and retransmission                                    | 14-15        |
| 5.4 Teardown                                                         | 16-17        |
| <b>6. Congestion-Control Results</b>                                 | <b>18-19</b> |
| 6.1 No loss: LOSS_PROBABILITY=0.0                                    | 18           |
| 6.2 Lossy run: LOSS_PROBABILITY=0.35                                 | 18-19        |
| 6.3 Separate lossy trial                                             | 19           |
| <b>7. Bonus: Fairness Under Competing Traffic and TCP comparison</b> | <b>20-22</b> |
| 7.1 Experiment setup (Experiment 1)                                  | 20           |
| 7.2 Fairness metric (Experiment 1)                                   | 20           |
| 7.3 Results (Experiment 1)                                           | 21           |
| 7.4 TCP Comparison (Experiment 2)                                    | 21-23        |
| <b>8. Conclusion</b>                                                 | <b>23</b>    |
| <b>References</b>                                                    | <b>23</b>    |

## 1. Introduction

---

This report describes our implementation of our pipelined Reliable Transfer Protocol. The task of this project was to build a connection-oriented, reliable, pipelined transport protocol as an application-layer service over UDP that included flow control and congestion control. UDP is deliberately minimal: it does not guarantee delivery, duplicate protection, ordering, retransmission, flow control, or congestion-control. This allows us to demonstrate these features in our own implementation [1]. The project therefore uses UDP only as the datagram carrier and places all reliability logic in `header.py`, `sender.py`, and `receiver.py`. Full source code, logs, and capture references throughout this report are available in our project repository [6].

The designed protocol provides the following:

- **Connection orientation:** A three-way handshake is required before data is sent, and a three-way FIN exchange is required to end the data stream.
- **Reliability:** A 16-bit checksum to detect corruption, sequence numbers, per-segment ACK's for accepted in-order data, and retransmission on timeout.
- **Pipelining:** Go-back-N with multiple segments being sent at once
- **Flow Control:** Each packet includes a receive window, which the sender respects.
- **Congestion Control:** Tahoe-inspired congestion control with slow start, congestion avoidance, and a return to slow start after timeout.
- **Loss and error simulation:** There's configurable packet dropping and corruption within the protocol to test the design on a loopback link.

## 2. Design and Implementation

---

### 2.1 Why UDP

We built our protocol over UDP (`SOCK_DGRAM`) because UDP does not provide reliable delivery, ordering, retransmission, flow control, or congestion control [1]. This allowed us to implement and observe these mechanisms ourselves rather than relying on TCP to provide them automatically. UDP therefore acts as the underlying unreliable channel, while reliability is handled by our protocol.

### 2.2 Custom segment format

Before implementing reliability, we designed a custom packet header containing the information required by our protocol. Every protocol message is represented by `PacketHeader` in `header.py`. The same structure is used for control messages and data. Sequence and acknowledgement number support reliable, ordered delivery, the advertised window supports flow control, and the checksum allows corrupted packets to be detected.

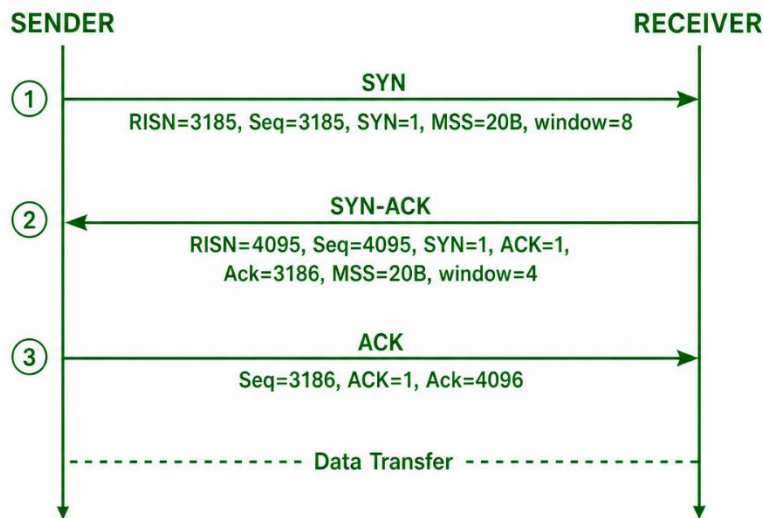
|                  |     |          |                  |     |     |
|------------------|-----|----------|------------------|-----|-----|
| Source Port      |     |          | Destination Port |     |     |
| Sequence Number  |     |          |                  |     |     |
| ACK Number       |     |          |                  |     |     |
| Window Size      | MSS | Checksum | SYN              | ACK | FIN |
| Application Data |     |          |                  |     |     |

In our implementation, these fields are serialized inside the UDP payload as `src,dst,seq,ack>window,mss,flags,checksum|payload`, with SYN, ACK and FIN represented as bits within a single flags value.

### 2.3 Connection establishment

To make the protocol connection-oriented, we use a three-message handshake before transmitting application data. The sender first sends a SYN containing a randomly generated initial sequence number. The receiver responds with a SYN-ACK containing its own, sequence number and acknowledging the sender's. Finally, the sender returns an ACK, confirming that both sides are ready for data transfer. This follows the synchronization idea used by TCP, while retaining our own project-specific sequence convention [2].

**Figure 1.0 3-way Handshake**



*Figure 1: Connection-establishment design used by the project: SYN, SYN-ACK, then ACK before data transfer*

### 2.4 Pipelining and flow control

We use a Go-Back-N (GBN) style approach for pipelined reliable delivery. The receiver accepts only the next expected sequence number and discards corrupted or out-of-order packets without acknowledging them. If the sender does not receive an ACK for the base packet before the timeout, recovery begins again from that base. This keeps the receiver simple because it does not need to buffer out-of-order packets. We chose this approach over Selective Repeat because it requires less receiver-side buffering and state, although out-of-order packets must be transmitted again later.

Flow control is handled separately using the receiver's advertised window. The sender limits the number of packets in flight using,  $\text{effective\_window} = \min(\text{sender window}, \text{CWND}/\text{MSS}, \text{receiver rwnd})$ . In our test setup, the sender window is 8 segments while the receiver advertises 4, so the receiver can restrict the sender even when the congestion window allows more.

Figure 2.0 Go-Back-N

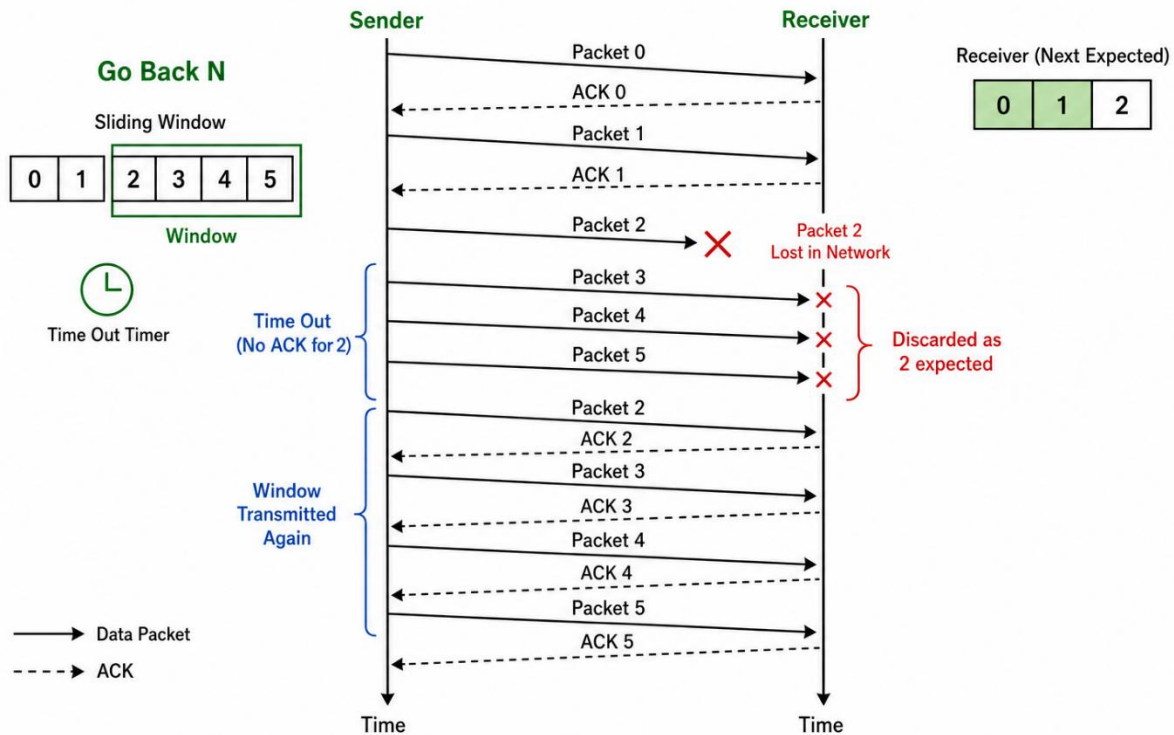


Figure 2: Go-Back-N concept used to motivate the project's in-order receiver and timeout recovery.

## 2.5 Congestion control – Tahoe-inspired, not full TCP Tahoe

We implemented TCP Tahoe-inspired congestion control using a congestion window (CWND) and slow-start threshold (ssthresh). CWND begins at one MSS. During our simplified Slow Start phase, it grows rapidly until the threshold is reached, after which the sender enters Congestion Avoidance and increases CWND more gradually. When loss is detected through a timeout, ssthresh is reduced and CWND is reset to one MSS before transmission begins growing again. This allows the sender to increase its transmission rate when the network is performing well and react conservatively when loss occurs [3].

### TCP Tahoe congestion control

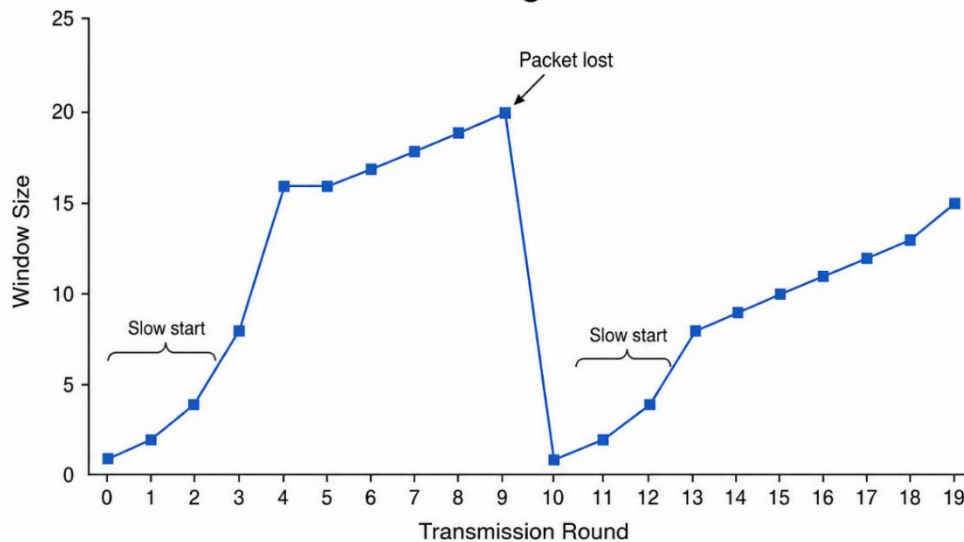


Figure 3.0 TCP Tahoe Sample

Figure 3: Conceptual Taheo-inspired behaviour used as the design reference: growth, loss, reset, and renewed growth

2.6 Adaptive Retransmission timeout

Instead of using one fixed timeout throughout the transfer, the sender estimates the retransmission timeout using measured RTT samples. The estimate is updated using smoothed RTT and RTT variation, and the resulting RTO is bounded between 0.4 and 8 seconds. After a timeout, the RTO is doubled to provide an exponential backoff. This allows the retransmission timer to adapt to observed network conditions rather than relying entirely on a fixed value.

2.7 Connection termination

Once all application data has been acknowledged, the sender gracefully terminated the connection. It first sends a FIN, the receiver replies with a combined FIN-ACK, and the sender completes the exchange with a final ACK. We combine the receiver’s FIN and ACK into one segment to keep the teardown simple while still explicitly confirming that both sides are finished.

Figure 4.0 FIN Exchange : Terminate Connection

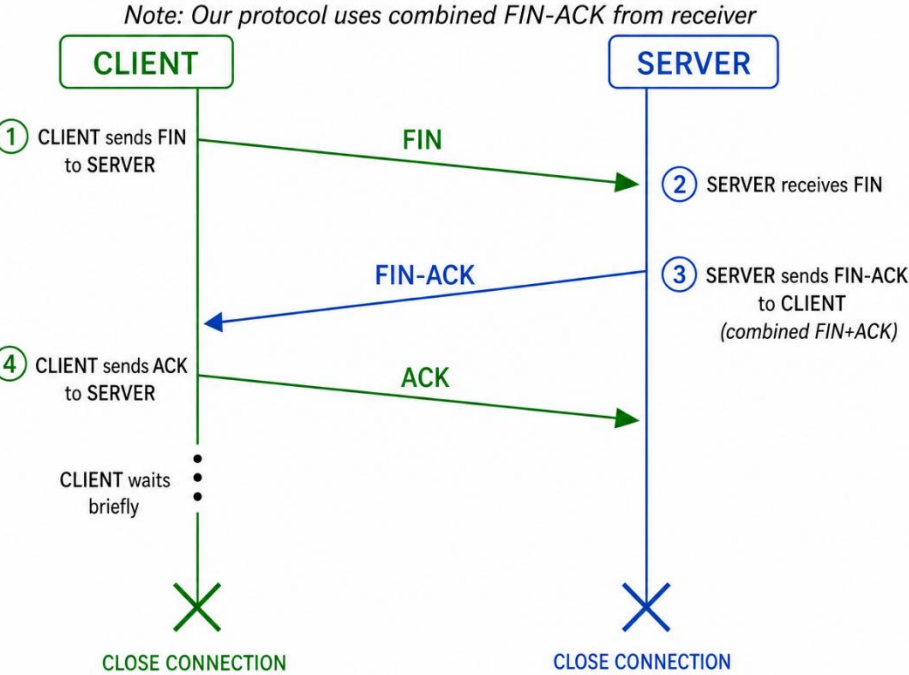


Figure 4. Project teardown: FIN, combined FIN-ACK, and final ACK.

2.8 Design Summary

To summarize, here are our chosen methodologies:

| Behaviour                | Mechanism                  | Design Reason                                     |
|--------------------------|----------------------------|---------------------------------------------------|
| Connection establishment | 3-way handshake            | Synchronizes both endpoints before data transfer  |
| Error detection          | 1’s complement checksum    | Detects corrupted segments                        |
| Pipelined reliability    | Go-Back-N-style delivery   | Simpler in-order receiver and timeout recovery    |
| Flow control             | Receiver-advertised window | Prevents the sender from overrunning the receiver |

|                    |                     |                                                  |
|--------------------|---------------------|--------------------------------------------------|
| Congestion control | Taheo-inspired CWND | Adjusts transmission rate in response to loss    |
| Timeout management | Adaptive RTO        | Adjusts retransmission timing using observed RTT |

### 3. Implementation Specifics

#### 3.1 Source Structure

| File                           | Responsibility                                                                                           |
|--------------------------------|----------------------------------------------------------------------------------------------------------|
| header.py                      | Manages the packets, fields, checksum and converting to and from bytes                                   |
| sender.py                      | Handles the client, handshake, splitting the file, send window, timers, congestion control and teardown. |
| receiver.py                    | Handles the Server, accepts the connection, checks packets while writing data in order and sending ACKs. |
| data.txt                       | Input application payload                                                                                |
| received_packets.txt           | Runtime reconstruction used to verify reliability                                                        |
| cwnd_log.csv / cwnd_growth.png | Congestion-control evidence generated by the sender                                                      |

#### 3.2 Sender algorithm

The sender tracks two positions in the sequence space: `base` and `next_seq`. The `base` is the oldest segment not yet acknowledged, and `next_seq` is the next new segment to send. It also keeps a record of what has been sent but not yet acknowledged to retransmit and measure the time of a packet round trip. This process can be described in three steps.

- **Step 1: Fill the window with information**

Send new segments until the limit is reached, taking into consideration the sender's own congestion control and flow control; whichever is smaller defines the cap. `min(WINDOW_SIZE, floor(CWND/MSS), receiver_window)`

- **Step 2: Wait for the ACK of base**

Ignore if anything fails on the checksum, comes from the wrong port, or acknowledges a different sequence number.

- **Step 3: React on an ACK**

Measure the round-trip time, update the timeout, grow the congestion window, and slide base forward. Loss is detected by timeout. The receiver does not emit duplicate ACKs, so there is no fast retransmit or fast recovery and loss recovery is timeout-driven. For timeout estimation, we use an RFC 6298-inspired adaptive RTO calculation [4]:

$$\text{EstimatedRTT} = 0.875 \times \text{EstimatedRTT} + 0.125 \times \text{SampleRTT}$$

$$\text{DevRTT} = 0.75 \times \text{DevRTT} + 0.25 \times |\text{SampleRTT} - \text{EstimatedRTT}|$$

$$\text{RTO} = \text{EstimatedRTT} + 4 \times \text{DevRTT}, \text{ (limited to 0.4–8 seconds)}$$

### 3.3 Receiver algorithm

The receiver keeps track of one variable: `expected_seq`. This is the next sequence number it will accept. Each packet that arrives is checked in this order:

1. **FIN flag set:** If it's true, start the teardown
2. **Checksum fails:** Drop the packet and send nothing back
3. **Wrong Sequence Number:** Drop the packet and send nothing back.

If none of this occurs, write the data, send an acknowledgment and move `expected_seq` forward.

By sending nothing, the receiver makes a corrupted packet appear as a lost packet. In this way, both corrupted and dropped packets are handled the same way when the sender's timer expires, and the packet is resent. With an expected sequence number, the data reaches the file in order automatically, and there's no need for sorting. Because of this, if there is ever an out of order segment the receiver never re-acknowledges, therefore a lost ACK would deadlock the transfer meaning the sender would retransmit a segment the receiver had already delivered, and the receiver would discard it in silence. Additionally, the advertised window is a constant at 4 segments, making our flow control a static bound.

### 3.4 Controlled loss and corruption

We implemented the loss and corruption simulation in `udt_send()`, the function in `sender.py` which wraps `socket.sendto()`. This function is the boundary between our protocol and the channel: everything above `udt_send()` handles the logic of our protocol, and everything below it handles the logic of the network. By doing this, the simulator becomes the unreliable channel, which ensures the sender's recovery path is exercised exactly as a genuinely lossy or corrupted link would exercise it. We considered instead discarding packets at the receiver on arrival but rejected it: that places test logic inside the protocol, and the sender's timer would then be reacting to a receiver decision rather than to a channel failure. Injecting at the sender's egress keeps the two concerns separate.

With a 35% probability, a data segment is either:

- **Dropped:** To achieve this, `sendto()` is never called, and the receiver doesn't see anything.
- **Corrupted:** To simulate a corrupted data segment, they are sent with a broken checksum; the receiver only sees a packet that fails the checksum check.

Each outcome occurs with equal probability, specifically 17.5%. Notably, packets carrying SYN, ACK, or FIN bypass the simulation entirely. Control packets carrying SYN, ACK, or FIN bypass our loss/corruption simulator. However, if these packets were lost by a real network, our handshake implementation does not retransmit them, so connection establishment could fail. Standard TCP includes retransmission handling for connection-establishment traffic [2][4].

### 3.5 Recovery behaviour after timeout

Whenever the timer runs out and the base is still unACKed, we implemented a recovery behaviour which follows three steps.

1. **The congestion window collapses.**  
 $SSTHRESH = \max(CWND / 2, 2 \times MSS)$   
 $CWND = MSS$   
Phase = slow start



The threshold becomes half the congestion window in use when the loss occurred or two segments, whichever is larger. The window drops to only one segment; from here the sender starts growing back from that point. Every change is logged to `cwnd_log.csv`; this file is used to produce the graphs in Section 6.

2. **The timer doubles:** By doubling the current RTO after a timeout, a sender is prevented from continuing to flood an already congested route. The RTO also has a minimum value of 0.4 seconds; on loopback, because the round-trip time is about 1 ms, this minimum is used for each retransmission and therefore each recovery takes at least 400 ms no matter what the measured RTT is.
3. **Retransmission begins from the base:** After a timeout, CWND is first reset to one MSS. The sender then recalculates its effective window and retransmits beginning from the base. Because CWND has just been reduced to one MSS, the first retransmission is only the base segment. Later missing segments are recovered as the base advances and may require additional timeouts.

This keeps the receiver simple because it does not buffer or reorder out-of-order segments. Compared with Selective Repeat, this requires less receiver-side state, although recovery can take longer because later missing segments may require additional timeouts.

## 4. Test Methodology and Reproducibility

---

For our testing, we wanted to confirm that the protocol we designed delivers the data correctly and to measure how the protocol's performance changes as the channel experiences overflow and more drops. This section will describe the setup and procedures used to perform the testing.

### 4.1 Environmental and default parameters

All the tests were run on a single machine over a loopback interface to ensure that the sender and receiver can communicate through the operating system network stack without the need for any physical link. This helps ensure that testing occurs in a controlled environment where the only loss and corruption are those we inject.

| Item          | Version/item             |
|---------------|--------------------------|
| Python        | 3.14.6                   |
| Interface     | Loopback(lo0), 127.0.0.1 |
| Capture Tool  | Wireshark / tshark 4.6.7 |
| Receiver port | 9000                     |
| Sender port   | 9001                     |
| Test Payload  | data.txt / 289 bytes     |

The test payload is divided into 15 segments with an MSS of 20 bytes. The small MSS helps make a small file that produces enough packets to show windowing, retransmission, and congestion behavior

We also set the protocol parameters as shown below in `sender.py` and `receiver.py`

| Parameter | Value    | Purpose                  |
|-----------|----------|--------------------------|
| MSS       | 20 bytes | Payload per data segment |

|                             |                  |                                                        |
|-----------------------------|------------------|--------------------------------------------------------|
| Sender Window Size          | 8 segments       | Maximum the sender is willing to send at a given time  |
| Receiver Window Size        | 4 segments       | Advertised receive window                              |
| Loss/Corruption Probability | 35%              | Set probability a data segment is dropped or corrupted |
| Alpha                       | 0.125            | Weight for EstimatedRTT                                |
| Beta                        | 0.25             | Weight for DevRTT                                      |
| RTO bounds                  | 0.4 – 8s         | Limits for the computed timeout                        |
| Initial timeout             | 1.5s             | Used before the first RTT sample exists                |
| Max Retries                 | 8                | FIN retransmission attempts                            |
| Initial CWND                | 20 bytes (1MSS)  | Starting congestion window                             |
| Initial SSTHRESH            | 80 bytes         | Slow start threshold before the first loss             |
| CWND_MAX                    | 160 bytes (8MSS) | Ceiling on the congestion window                       |

It's important to clarify that the window sizes are intentionally different: the receiver advertises 4, and the sender advertises 8. During early Slow Start, CWND can be the binding constraint; once CWND permits four or more segments, the receiver's advertised window of 4 becomes the flow-control cap. This makes the flow-control effective when looking at the packet capture.

## 4.2 Run procedures

To perform a basic run, the receiver starts listening before the sender's SYN arrives. Each side runs in its own terminal. The receiver then writes every accepted payload to `received_packets.txt`. Following this, the sender writes its congestion window history to `cwnd_log.csv` and, using this file, plots it in `cwnd_growth.png` upon completion. To capture traffic, we used a loopback interface before the run was started (macOS):

```
tshark -i lo0 -f "udp port 9000 or udp port 9001" -w capture.pcapng
```

Within the Wireshark GUI, we select the loopback interface and apply the display filter.

```
udp.port == 9000 || udp.port == 9001
```

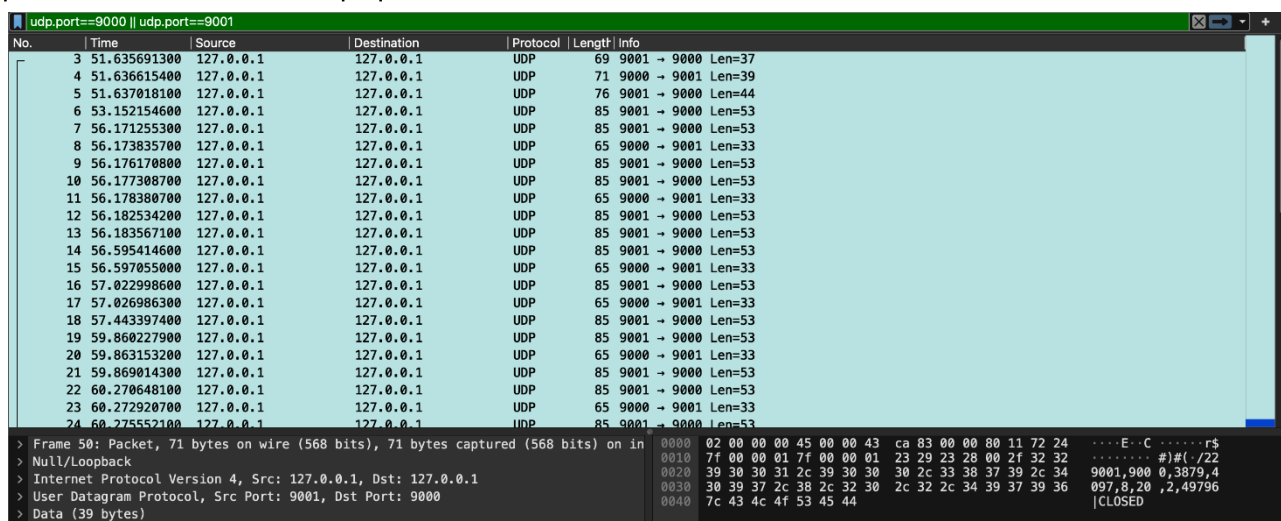


Figure 5. Wireshark with the filter activated

### 4.3 Test matrix

We measured five simulated loss/corruption probabilities with 5 trials each, giving us a total of 25 transfers. Each trial required a complete connection: handshake, full file transfer, and teardown, giving this test matrix:

| Loss/Corruption Prob | Purpose                                                                                        |
|----------------------|------------------------------------------------------------------------------------------------|
| 0%                   | Baseline test, to establish best case transfer time with no retransmission                     |
| 2%                   | Light Loss. The congestion windows can still grow through slow start and congestion avoidance. |
| 10%                  | Moderate Loss, Regular timeouts and the window repeatedly reset.                               |
| 20%                  | Heavy loss. The transfer time is mostly recovery                                               |
| 35%                  | Stress test, beyond realistic network conditions                                               |

### 4.4 End-to-end correctness check

To ensure the performance numbers we obtained from the test were meaningful, every trial performed was verified to ensure the data was intact.

To check this, we performed a direct byte-for-byte comparison between the original data.txt and the receiver's received\_packets.txt. By comparing, we ensure that any single wrong, missing, duplicated, or reordered byte is caught wherever it occurs in the file, which is better than performing a visual inspection. A trial was considered successful only when we confirmed the two files were identical; in any other case, it was counted as a failure (e.g., files that produced different output or transfers that were not completed).

The overall results of our protocol are shown in the following table:

| Loss/Corruption Prob | Avg Time | Min     | Max    | Throughput | Avg Retransmissions | Correct |
|----------------------|----------|---------|--------|------------|---------------------|---------|
| 0%                   | 0.4115s  | 0.3256s | 0.627s | 702.4 B/s  | 0.0                 | 5/5     |
| 2%                   | 0.7318s  | 0.3318s | 2.301s | 394.9 B/s  | 0.4                 | 5/5     |
| 10%                  | 5.429s   | 1.879s  | 8.421s | 53.24 B/s  | 10.0                | 5/5     |
| 20%                  | 6.633s   | 4.53s   | 10.81s | 43.57 B/s  | 12.2                | 5/5     |
| 35%                  | 13.99s   | 8.826s  | 22.87s | 20.66 B/s  | 19.4                | 5/5     |

One thing to highlight from this table is the 2% row, where a single trial took 2.30 s after two retransmissions while the other four completed in 0.33–0.35 s. The degradation we observe at higher impairment probabilities is largely attributable to repeated timeout-based recovery. Our RTO has a minimum of 0.4 s, but it is dynamic: it can back off after a timeout and later be recomputed from RTT measurements. Therefore, we cannot assign a fixed cost of 0.4 s to every retransmission. The safe conclusion is that repeated timeout waits are a major reason transfer time increases as the impairment probability rises.

```

loss=0.02 trial 1: 2.30s retransmits=2 correct=True
loss=0.02 trial 2: 0.33s retransmits=0 correct=True
loss=0.02 trial 3: 0.34s retransmits=0 correct=True
loss=0.02 trial 4: 0.34s retransmits=0 correct=True
loss=0.02 trial 5: 0.35s retransmits=0 correct=True
-> loss=0.02 avg= 0.73s min= 0.33 max= 2.30 394.9 B/s rexmit= 0.4 correct=5/5

```

Figure 6. Results of the five trials with 2% L/C rate, as listed on benchmark\_output.txt

## 5. Traffic Analysis – Wireshark Evidence

The Wireshark filter `udp.port == 9000 or udp.port == 9001` isolated the custom protocol traffic. Wireshark still labels the transport as UDP because our reliability header sits inside the UDP payload. The readable serialization is useful here: sequence number, ACK, window, MSS, flags, checksum, and short control labels can all be inspected in the payload pane.

### 5.1 Three-way handshake

The complete handshake takes approximately 1.327 ms on loopback. The trace also shows the intended flow-control asymmetry immediately: the sender advertises 8 while the receiver advertises 4.

The image shows a Wireshark packet capture of a SYN packet. The packet list shows a packet from 127.0.0.1:9001 to 127.0.0.1:9000. The packet details pane shows the User Datagram Protocol and the payload. The payload is a SYN packet with sequence number 3576, window 8, MSS 20, and the flag CONNECT.

| No. | Time         | Source    | Destination | Protocol | Length | Info               |
|-----|--------------|-----------|-------------|----------|--------|--------------------|
| 3   | 51.635691300 | 127.0.0.1 | 127.0.0.1   | UDP      | 69     | 9001 → 9000 Len=37 |
| 4   | 51.636615400 | 127.0.0.1 | 127.0.0.1   | UDP      | 71     | 9000 → 9001 Len=39 |
| 5   | 51.637018100 | 127.0.0.1 | 127.0.0.1   | UDP      | 76     | 9001 → 9000 Len=44 |
| 6   | 53.152154600 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 7   | 56.171255300 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 8   | 56.173835700 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 9   | 56.176170800 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 10  | 56.177308700 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 11  | 56.178380700 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 12  | 56.182534200 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 13  | 56.183567100 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 14  | 56.595414600 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 15  | 56.597055000 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 16  | 57.022998600 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 17  | 57.026986300 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 18  | 57.443397400 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 19  | 59.860227900 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 20  | 59.863153200 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 21  | 59.869014300 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 22  | 60.270648100 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 23  | 60.272920700 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 24  | 60.275552100 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 25  | 60.687811800 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |

Packet 3 details:

- Identification: 0xca54 (51796)
- ... .. = Flags: 0x0
- ...0 0000 0000 0000 = Fragment Offset: 0
- Time to Live: 128
- Protocol: UDP (17)
- Header Checksum: 0x7255 [validation disabled]
- [Header checksum status: Unverified]
- Source Address: 127.0.0.1
- Destination Address: 127.0.0.1
- [Stream index: 1]
- User Datagram Protocol, Src Port: 9001, Dst Port: 9000
- Source Port: 9001
- Destination Port: 9000
- Length: 45
- Checksum: 0xa127 [unverified]
- [Checksum Status: Unverified]
- [Stream index: 0]
- [Stream Packet Number: 1]
- [Timestamps]
- UDP payload (37 bytes)
- Data (37 bytes)
- Data: 393030312c393030302c333537362c302c382c32302c342c33323934397c434f4e4e454551
- [Length: 37]

Packet 3 hex dump:

```

0000 02 00 00 00 45 00 00 41 ca 54 00 00 80 11 72 55 ... E . A . T . . . r U
0010 7f 00 00 01 7f 00 00 01 23 29 23 28 00 2d a1 27 ... . . . . #) ( . . .
0020 39 30 30 31 2c 39 30 30 30 2c 33 35 37 36 2c 30 9001,900 0,3576,0
0030 2c 38 2c 32 30 2c 34 2c 33 32 39 34 39 7c 43 4f ,8,20,4, 32949[CO
0040 4e 4e 45 43 54 NNECT

```

Figure 7. Wireshark packet 3: the sender SYN. The payload visibility contains port 9001 ->9000, seq 3576, window 8, MSS 20, flags 4, and CONNECT.

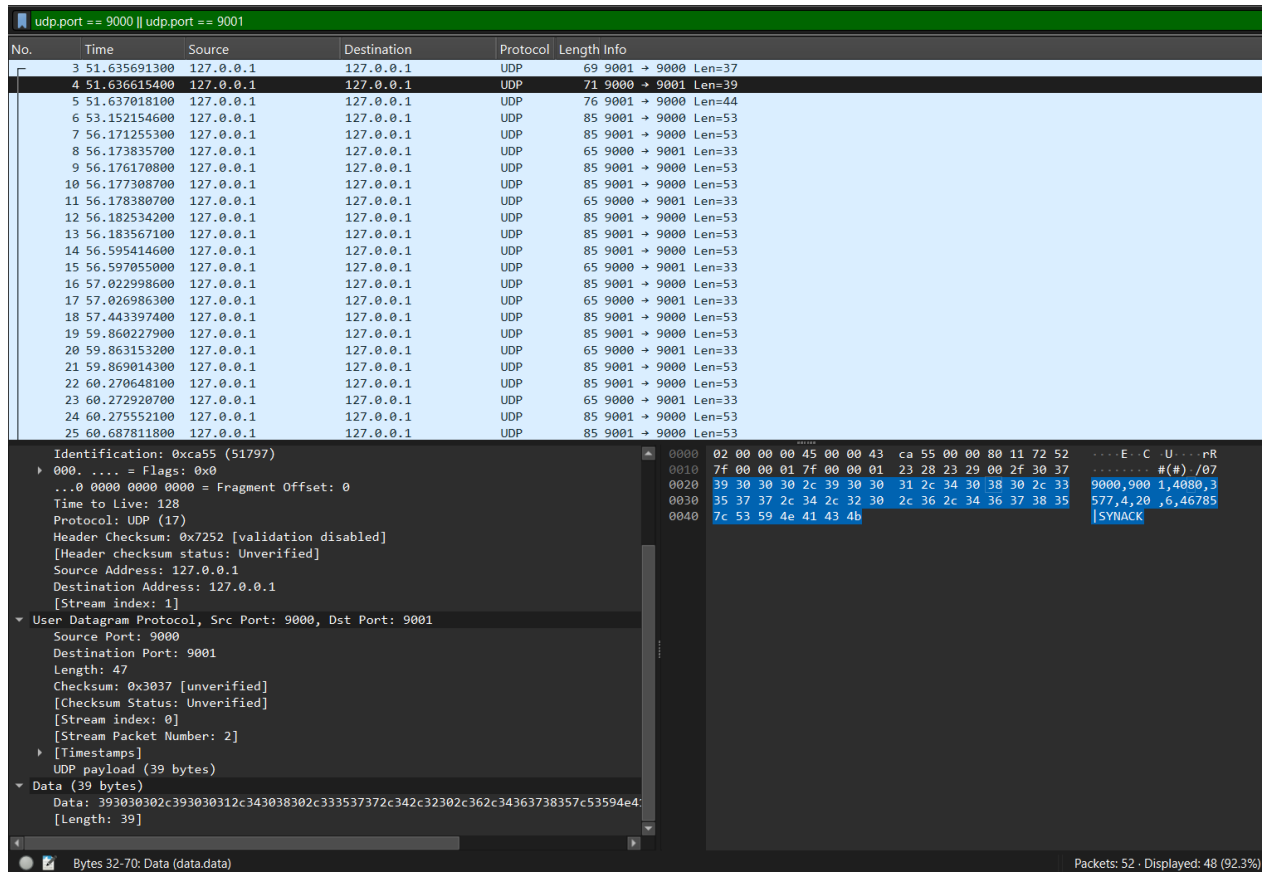


Figure 8. Wireshark packet 4: receiver SYN-ACK with receiver window 4 and flags value 6 (SYN + ACK)

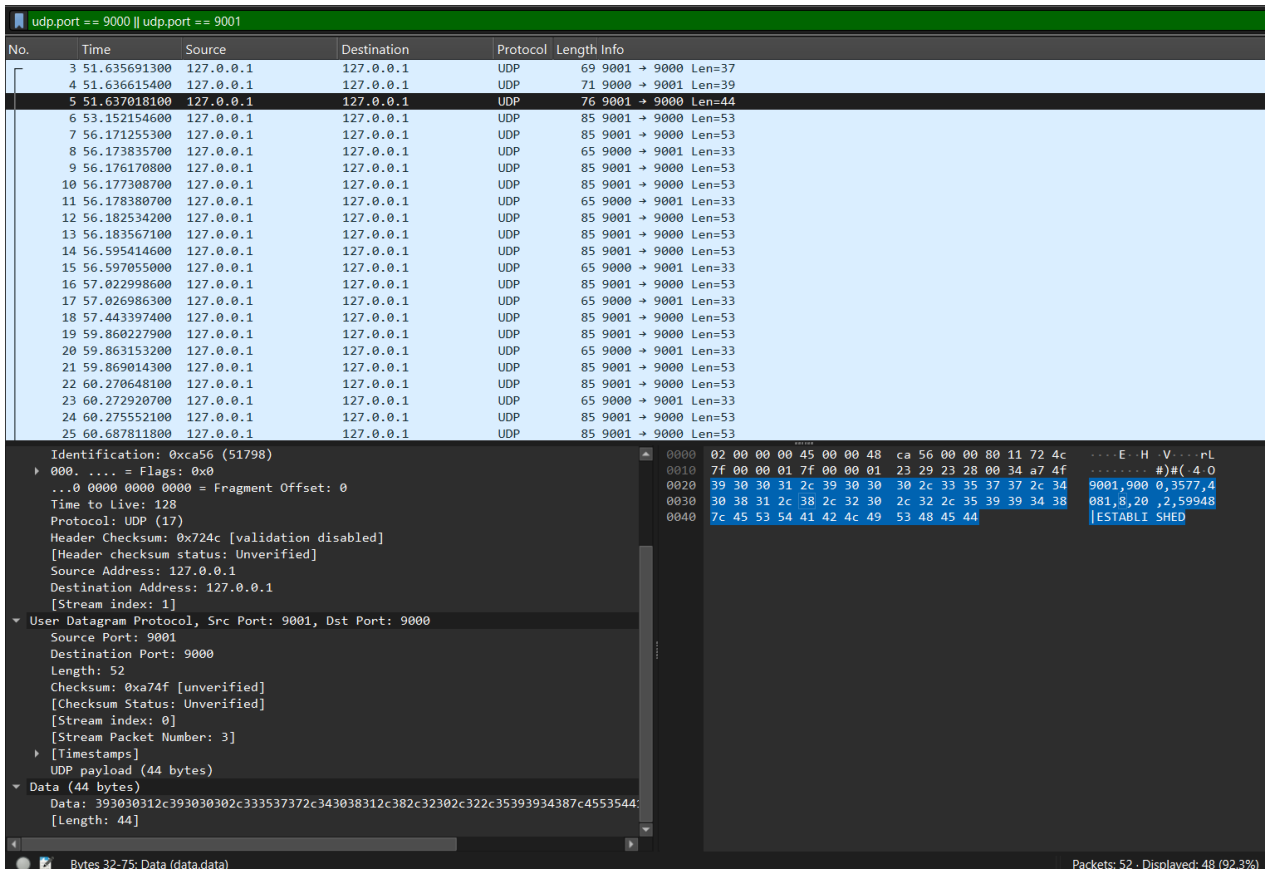


Figure 9. Wireshark packet 5: final ACK with ESTABLISHED, completing connection setup before application data.

## 5.2 Flow-control evidence

Frame 8 is a receiver ACK after the first successful data delivery. The custom payload reads 9000,9001,4081,3578,4,20,2, ... The window field is therefore 4, and the ACK field is 3578. sender.py stores that advertised value and applies it inside `effective_window()`.

| No. | Time         | Source    | Destination | Protocol | Length | Info               |
|-----|--------------|-----------|-------------|----------|--------|--------------------|
| 3   | 51.635691300 | 127.0.0.1 | 127.0.0.1   | UDP      | 69     | 9001 → 9000 Len=37 |
| 4   | 51.636615400 | 127.0.0.1 | 127.0.0.1   | UDP      | 71     | 9000 → 9001 Len=39 |
| 5   | 51.637018100 | 127.0.0.1 | 127.0.0.1   | UDP      | 76     | 9001 → 9000 Len=44 |
| 6   | 53.152154600 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 7   | 56.171255300 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 8   | 56.173835700 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 9   | 56.176170800 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 10  | 56.177308700 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 11  | 56.178380700 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 12  | 56.182534200 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 13  | 56.183567100 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 14  | 56.595414600 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 15  | 56.597055000 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 16  | 57.022998600 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 17  | 57.026986300 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 18  | 57.443397400 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 19  | 59.860227900 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 20  | 59.863153200 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 21  | 59.869014300 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 22  | 60.270648100 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 23  | 60.272920700 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 24  | 60.275552100 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 25  | 60.687811800 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |

|                                                                          |                                                      |                  |
|--------------------------------------------------------------------------|------------------------------------------------------|------------------|
| ...0 0000 0000 0000 = Fragment Offset: 0                                 | 0000 02 00 00 00 45 00 00 3d ca 59 00 00 80 11 72 54 | ...E..=..Y...rT  |
| Time to Live: 128                                                        | 0010 7f 00 00 01 7f 00 00 01 23 28 23 29 00 29 16 27 | .....#(##).'     |
| Protocol: UDP (17)                                                       | 0020 39 30 30 30 2c 39 30 30 31 2c 34 30 38 31 2c 33 | 9000,9001,4081,3 |
| Header Checksum: 0x7254 [validation disabled]                            | 0030 35 37 38 2c 34 2c 32 30 2c 32 2c 33 39 38 34 39 | 578,4,20,2,39849 |
| [Header checksum status: Unverified]                                     | 0040 7c                                              |                  |
| Source Address: 127.0.0.1                                                |                                                      |                  |
| Destination Address: 127.0.0.1                                           |                                                      |                  |
| [Stream index: 1]                                                        |                                                      |                  |
| ▼ User Datagram Protocol, Src Port: 9000, Dst Port: 9001                 |                                                      |                  |
| Source Port: 9000                                                        |                                                      |                  |
| Destination Port: 9001                                                   |                                                      |                  |
| Length: 41                                                               |                                                      |                  |
| Checksum: 0x1627 [unverified]                                            |                                                      |                  |
| [Checksum Status: Unverified]                                            |                                                      |                  |
| [Stream index: 0]                                                        |                                                      |                  |
| [Stream Packet Number: 6]                                                |                                                      |                  |
| ▼ [Timestamps]                                                           |                                                      |                  |
| [Time since first frame: 4.538144400 seconds]                            |                                                      |                  |
| [Time since previous frame: 2.580400 milliseconds]                       |                                                      |                  |
| UDP payload (33 bytes)                                                   |                                                      |                  |
| ▼ Data (33 bytes)                                                        |                                                      |                  |
| Data: 393030302c393030312c343038312c333537382c342c32302c322c33393834397c |                                                      |                  |
| [Length: 33]                                                             |                                                      |                  |

Figure 10. Receiver ACK advertising `rwnd` = 4. This value constrains the sender even though its configured window is 8.

### Why this proves flow control

The limit is not merely printed in the header. sender.py calculates `min(WINDOW_SIZE, floor(CWND/MSS), receiver_rwnd)`, so a receiver-advertised 4 can actively cap the number of data segments in flight.

## 5.3 Corruption and retransmission

The strongest trace evidence is the repeated first data transfer, 3578. Wireshark packet No. 6 at 53.1521546 s carries the first 20-byte payload (“Among the quiet star...”) but its injected custom checksum is one larger than the correct value. The receiver rejects it. Wireshark packet No. 7 at 56.1712553 s carries the same sequence and payload with the corrected checksum.



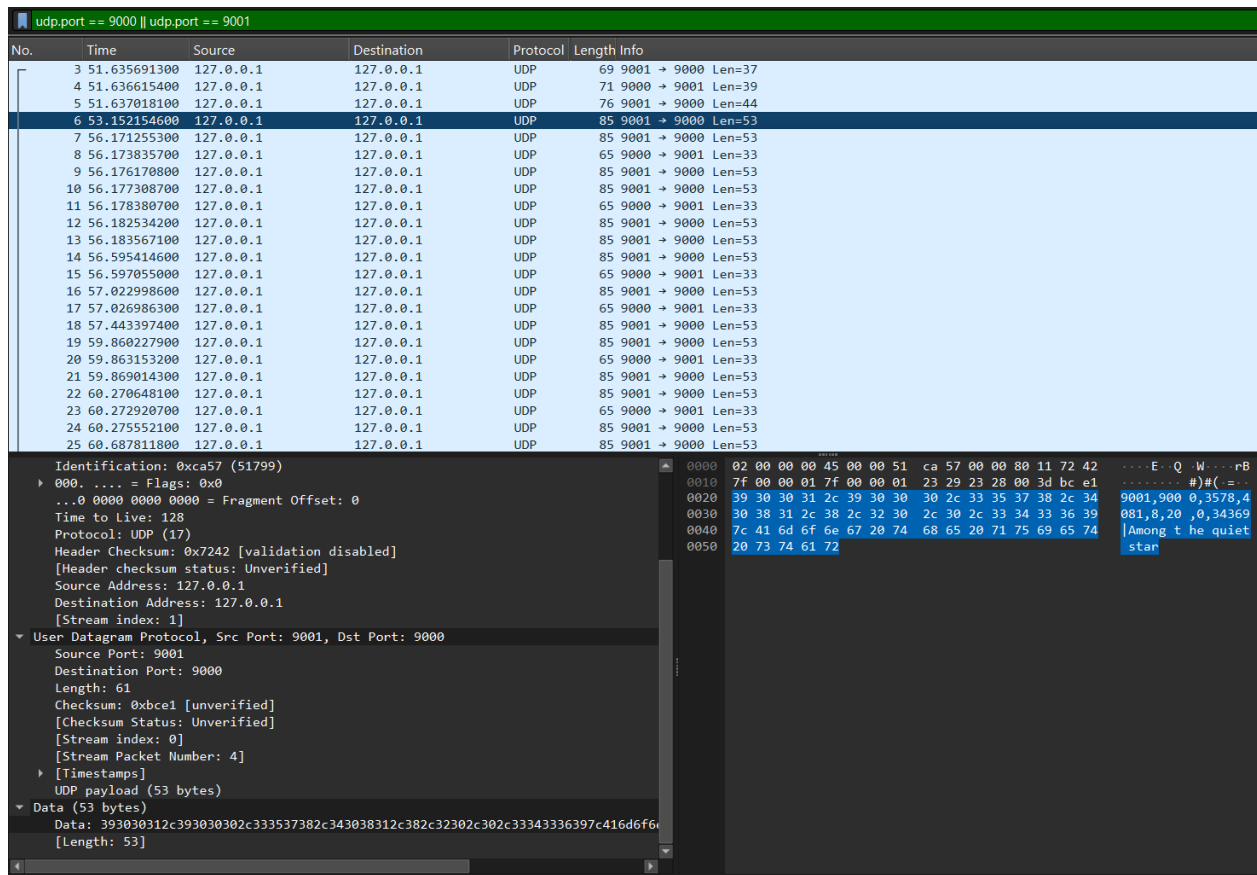


Figure 11. First visible transmission of sequence 3578. The deliberately altered custom checksum causes receiver-side rejection.

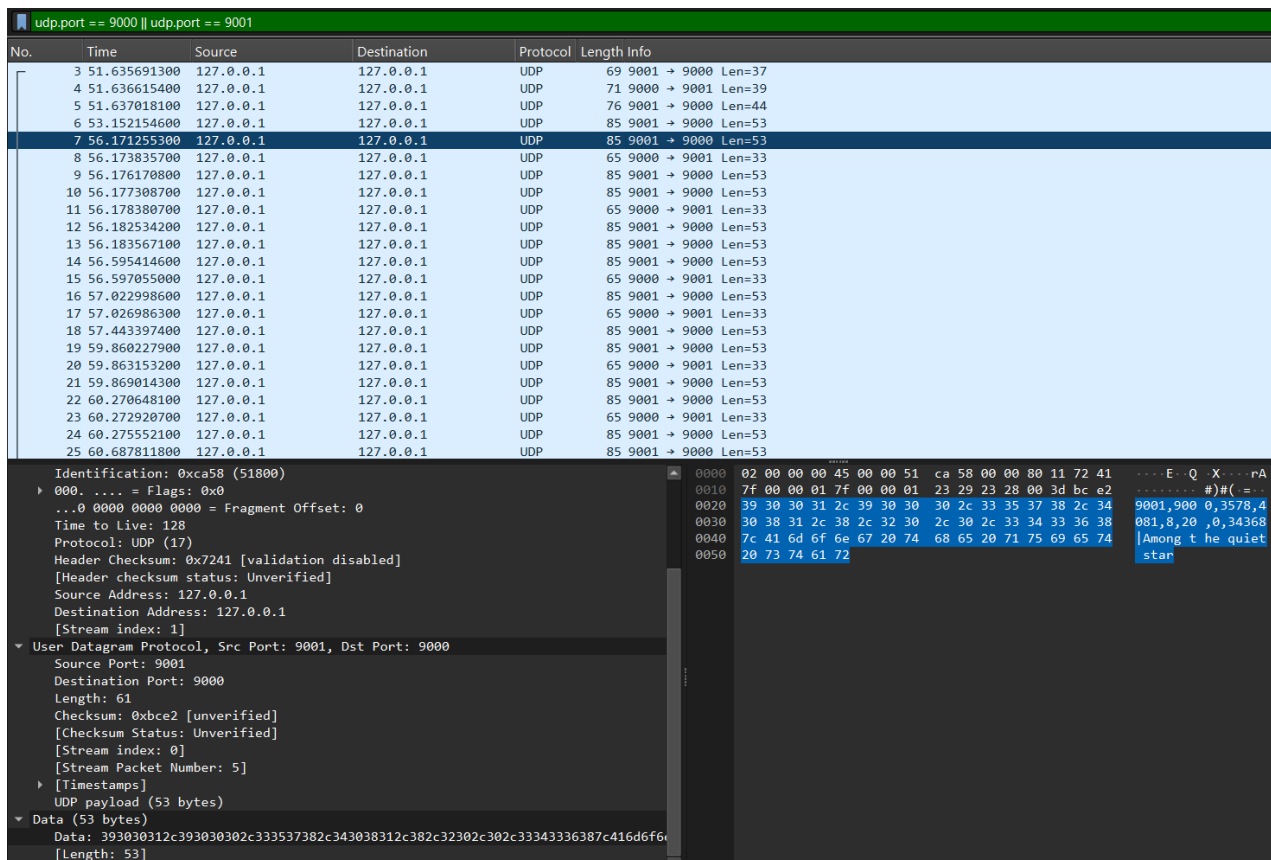


Figure 12. Sequence 3578 is transmitted again with the corrected checksum after timeout; frame 8 then ACKs the accepted sequence.

## 5.4 Teardown

Frames 48-50 form the closing exchange. Frame 48 at 64.8521652s is a sender FIN; its custom payload includes seq=3878, flags=1, and BYE. The next two diagrams reverse direction for FIN-ACK and then return for the final ACK. From FIN to final ACK, teardown takes about 1.468ms on loopback.

| udp.port == 9000    udp.port == 9001 |              |           |             |          |        |                    |
|--------------------------------------|--------------|-----------|-------------|----------|--------|--------------------|
| No.                                  | Time         | Source    | Destination | Protocol | Length | Info               |
| 29                                   | 61.104099600 | 127.0.0.1 | 127.0.0.1   | UDP      | 84     | 9001 → 9000 Len=52 |
| 30                                   | 61.513006900 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 31                                   | 61.515481800 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 32                                   | 61.517852200 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 33                                   | 61.928134200 | 127.0.0.1 | 127.0.0.1   | UDP      | 84     | 9001 → 9000 Len=52 |
| 34                                   | 61.931142400 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 35                                   | 61.933850400 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 36                                   | 63.158565000 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 37                                   | 63.164519700 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 38                                   | 63.169580000 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 39                                   | 63.576702000 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 40                                   | 63.580521200 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 41                                   | 63.992675800 | 127.0.0.1 | 127.0.0.1   | UDP      | 85     | 9001 → 9000 Len=53 |
| 42                                   | 63.995659900 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 43                                   | 63.997841500 | 127.0.0.1 | 127.0.0.1   | UDP      | 74     | 9001 → 9000 Len=42 |
| 44                                   | 64.412854600 | 127.0.0.1 | 127.0.0.1   | UDP      | 84     | 9001 → 9000 Len=52 |
| 45                                   | 64.423942800 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 46                                   | 64.843950500 | 127.0.0.1 | 127.0.0.1   | UDP      | 74     | 9001 → 9000 Len=42 |
| 47                                   | 64.847394400 | 127.0.0.1 | 127.0.0.1   | UDP      | 65     | 9000 → 9001 Len=33 |
| 48                                   | 64.852165200 | 127.0.0.1 | 127.0.0.1   | UDP      | 67     | 9001 → 9000 Len=35 |
| 49                                   | 64.853094300 | 127.0.0.1 | 127.0.0.1   | UDP      | 71     | 9000 → 9001 Len=39 |
| 50                                   | 64.853633000 | 127.0.0.1 | 127.0.0.1   | UDP      | 71     | 9001 → 9000 Len=39 |

...0 0000 0000 0000 = Fragment Offset: 0  
Time to Live: 128  
Protocol: UDP (17)  
Header Checksum: 0x722a [validation disabled]  
[Header checksum status: Unverified]  
Source Address: 127.0.0.1  
Destination Address: 127.0.0.1  
[Stream index: 1]  
▼ User Datagram Protocol, Src Port: 9001, Dst Port: 9000  
Source Port: 9001  
Destination Port: 9000  
Length: 43  
Checksum: 0x1374 [unverified]  
[Checksum Status: Unverified]  
[Stream index: 0]  
[Stream Packet Number: 46]  
▼ [Timestamps]  
[Time since first frame: 13.216473900 seconds]  
[Time since previous frame: 4.770800 milliseconds]  
UDP payload (35 bytes)  
▼ Data (35 bytes)  
Data: 393030312c393030302c333837382c343038312c382c32302c312c343839377c425945  
[Length: 35]

0000 02 00 00 00 45 00 00 3f ca 81 00 00 80 11 72 2a ... E...? .....r\*  
0010 7f 00 00 01 7f 00 00 01 23 29 23 28 00 2b 13 74 .....#)(. +t  
0020 39 30 30 31 2c 39 30 30 30 2c 33 38 37 38 2c 34 9001,900 0,3878,4  
0030 30 38 31 2c 38 2c 32 30 2c 31 2c 34 38 39 37 7c 001,8,20 ,1,4897|  
0040 42 59 45 BYE

Figure 13. Start of the captured teardown. Frame 48 carries the custom FIN (flags=1, BYE)



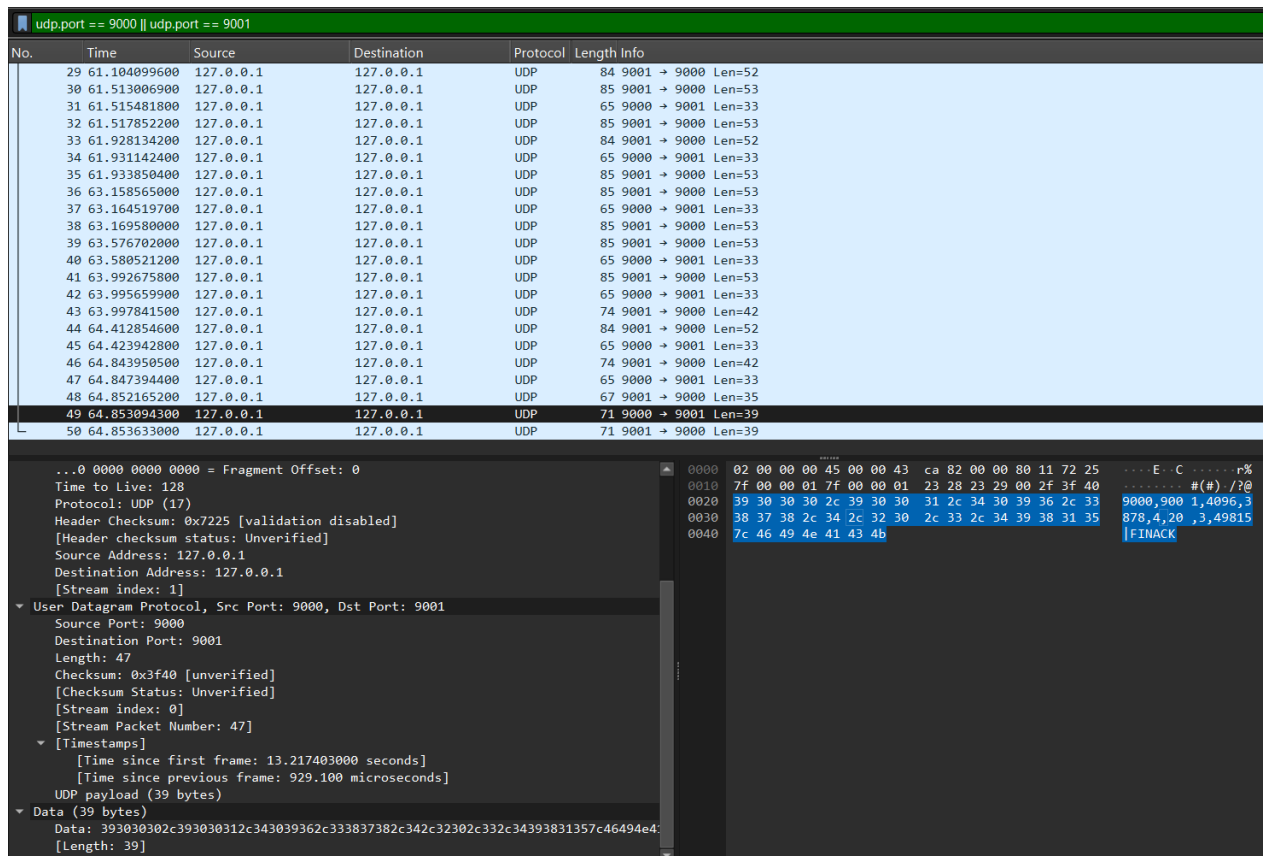


Figure 14. Frame 49: Receiver sends a FIN-ACK and terminated connection

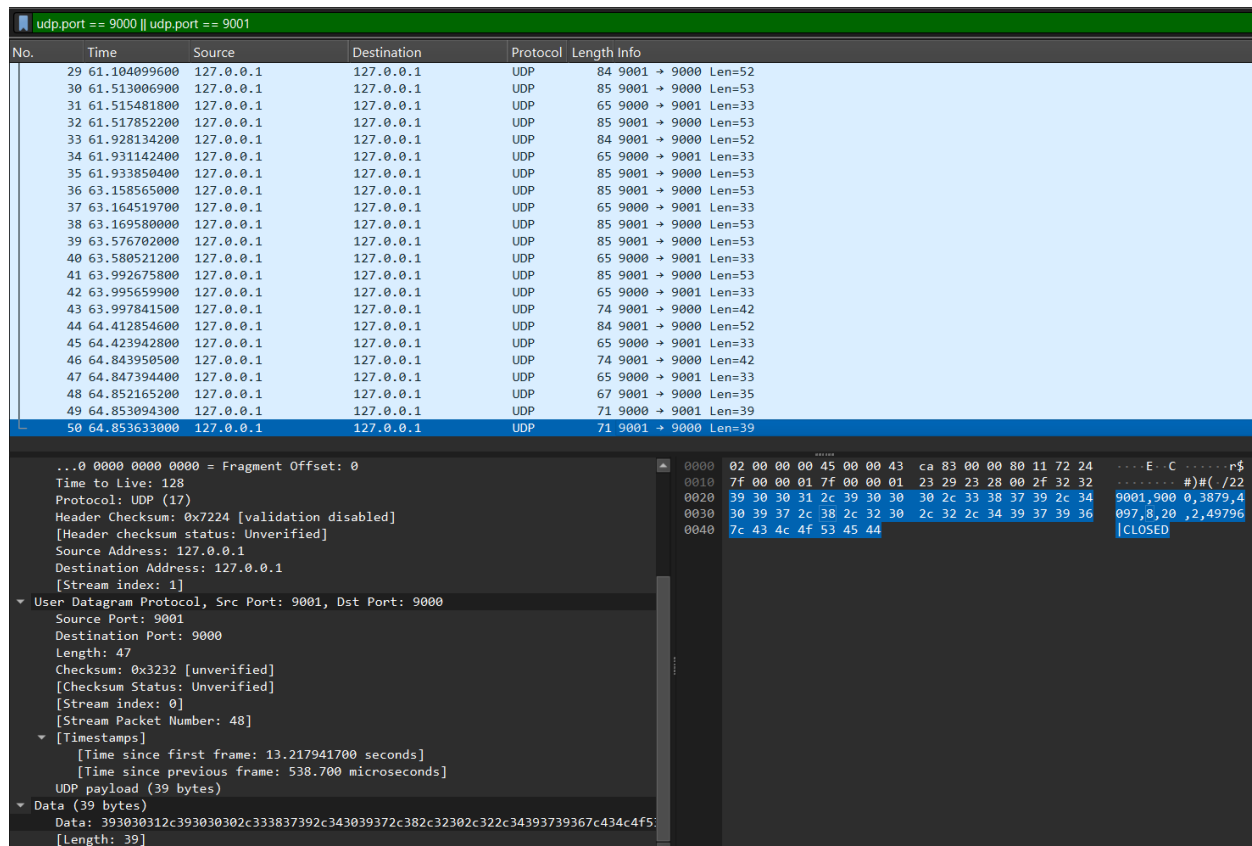


Figure 15. Frame 50: Sender sends ACK and closes the connection.

## 6. Congestion-Control Results

### 6.1 No-loss: LOSS\_PROBABILITY = 0.0

With no simulated loss, the congestion window increases rapidly during **Slow Start** until it reaches the slow-start threshold (ssthresh). The sender then transitions to **Congestion Avoidance**, where CWND increases more gradually. Since no timeout occurs, CWND is not reset during the transfer and eventually reaches the configured maximum.

| No-loss metric                | Observed              |
|-------------------------------|-----------------------|
| Timeout events                | 0                     |
| ACK growth events             | 15                    |
| CWND start                    | 20 bytes              |
| CWND maximum                  | 160 bytes (8 MSS cap) |
| Initial ssthresh              | 80 bytes              |
| Logged data-transfer interval | 0.0254s on loopback   |

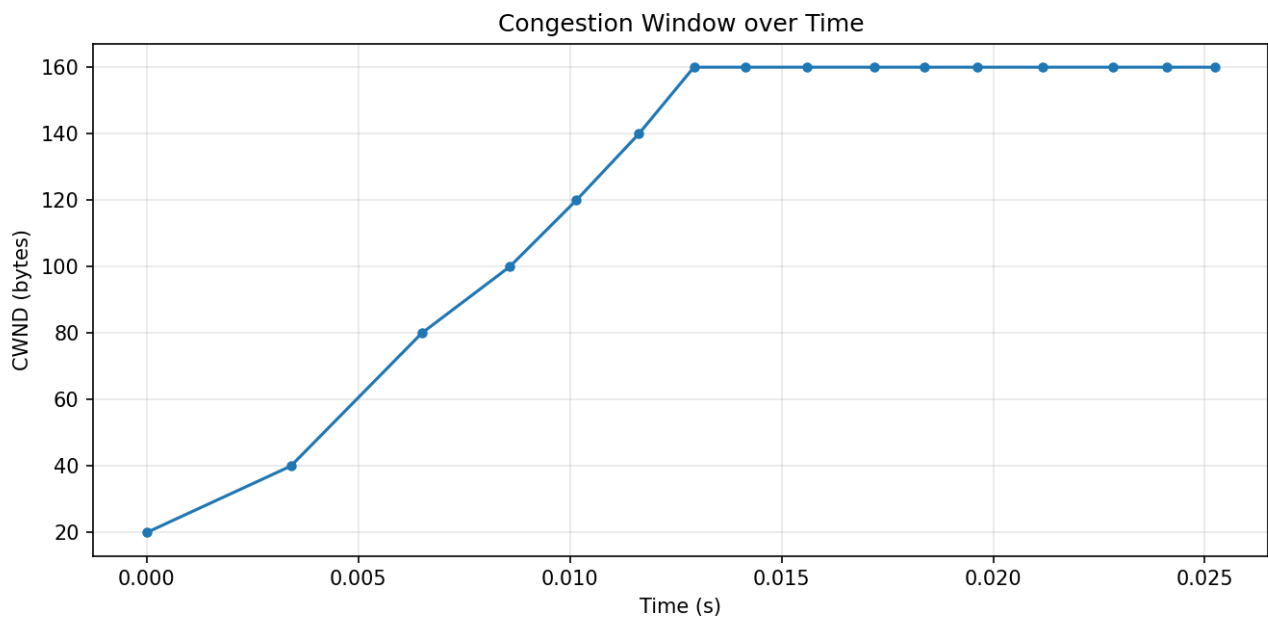


Figure 16. No-loss control run. CWND rises from 20 to the configured 160-byte cap; rwnd still limits actual flight size to four segments.

### 6.2 Lossy run: LOSS\_PROBABILITY = 0.35

With the simulated loss/corruption probability increased to 35%, multiple packet-loss and corruption events occur during transmission. When the sender does not receive the required ACK before the timeout, it treats the event as congestion, reduces ssthresh, resets CWND to 1 MSS (20 bytes), and returns to Slow Start. As successful ACKs are received, CWND begins growing again until another loss event occurs. The repeated growth and sharp drops in the graph demonstrate that the sender reduces its transmission rate after loss and gradually increases it again when acknowledgments resume.

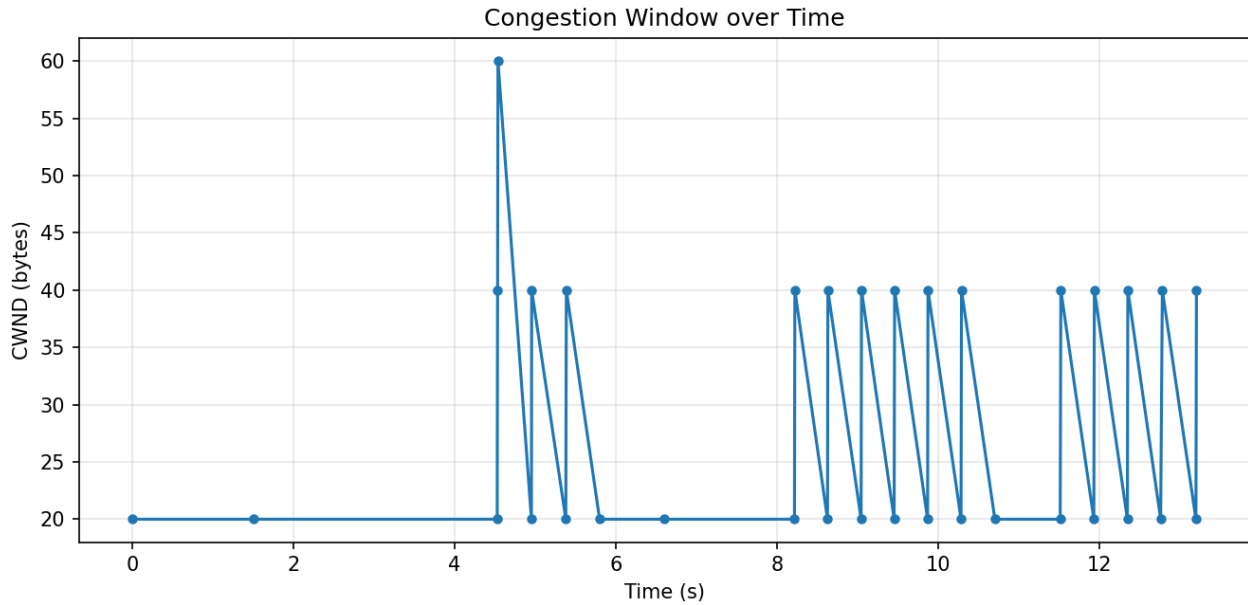


Figure 17. Supplied lossy-run CWND trace. Repeated timeout events reset CWND to one MSS; short recovery periods reach 40-60 bytes before the next loss.

### 6.3 Separate lossy trial

A second supplied lossy plot shows a different random path: CWND briefly climbs as high as 140 bytes before a loss resets it to 20. After that, repeated losses hold the window mostly between 20 and 40 bytes. The difference between the two plots is expected because the impairment decision is random on every pure data transmission.

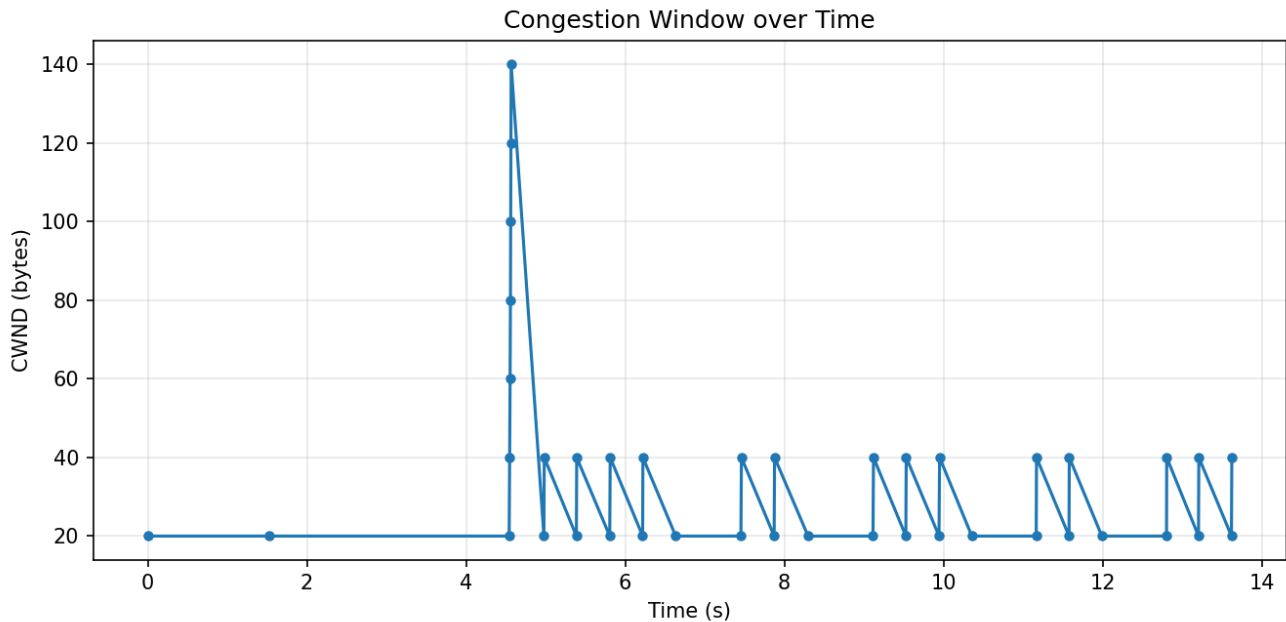


Figure 18. Separate lossy trial. A longer initial growth period reaches 140 bytes before timeout recovery resets the congestion window.

**Interpretation:** Together, the clean and lossy runs show the two behaviours: the sender can open its window when ACKs arrive, and it becomes conservative again when a timeout signals loss.

## 7. Bonus - Fairness Under Competing Traffic and TCP comparison

### Experiment 1: Fairness Under Competing Traffic

To answer whether our protocol shares a constrained resource fairly when another flow is active at the same time, we realized that running two flows directly on unconstrained loopback would not create meaningful competition, so we routed both protocol flows through one finite user-space UDP bottleneck. The two senders therefore compete for the same data queue, while control and return ACK traffic is forwarded immediately.

#### 7.1 Experiment setup (Experiment 1)

| Element                      | Configuration                                                                |
|------------------------------|------------------------------------------------------------------------------|
| Flow A                       | Sender 9001 -> relay 9100 -> receiver 9200                                   |
| Flow B                       | Sender 9003 -> relay 9102 -> receiver 9202                                   |
| Shared bottleneck            | One FIFO queue for data packets from both flows                              |
| Queue limit                  | 4 data datagrams                                                             |
| Service interval             | 35ms per forwarded DATA datagram                                             |
| Per-sender random impairment | Disabled (LOSS_PROBABILITY=0.0) so contention is created by the shared queue |
| Payload                      | 4096 bytes per flow (~205 segments at MSS= 20)                               |
| Trials                       | 5 long-payload trials                                                        |

The relay is a test harness only; it does not change the reliability or congestion-control logic in sender.py and receiver.py. Pure DATA packets from both flows enter the same FIFO queue. If the queue is full, an arriving DATA packet is dropped. Control traffic and returning ACKs are forwarded immediately, so the measured contention is concentrated on the shared DATA path. We also disabled the sender's random loss/corruption simulator for these trials so that losses came from the common bottleneck rather than from independent per-flow randomness.

We first used a 289-byte smoke test, then repeated the fairness experiment with 4096-byte payloads so each flow remained active for roughly 205 application segments. Five trials were used to check whether an apparently fair result was repeatable rather than a one-run coincidence.

#### 7.2 Fairness metric (Experiment 1)

We quantify sharing with Jain's fairness index [5]. For the two measured throughputs  $x_A$  and  $x_B$ :

$$J = (x_A + x_B)^2 / (2(x_A^2 + x_B^2))$$

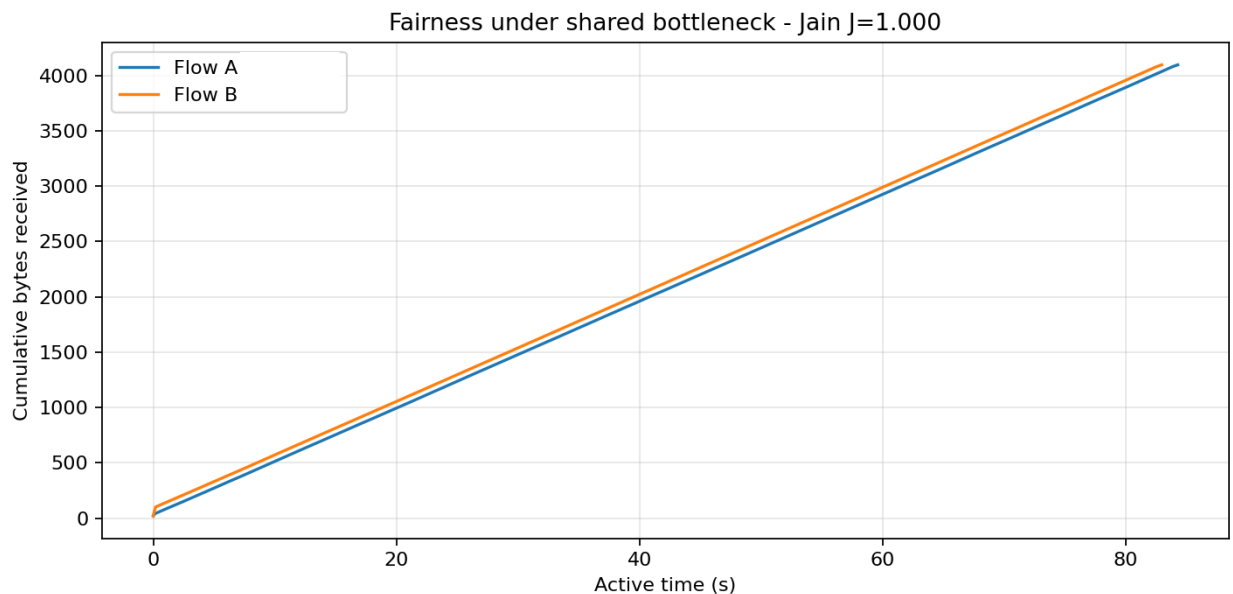
The index is bounded between 0 and 1; values closer to 1 indicate more equal allocation. Reporting the numeric index alongside per-flow throughput makes the fairness claim testable instead of relying only on similar-looking traces.

### 7.3 Results (Experiment 1)

| Trial | Flow A (B/s) | Flow B (B/s) | A time (s) | B time (s) | Jain J   |
|-------|--------------|--------------|------------|------------|----------|
| 1     | 48.51        | 49.25        | 84.44      | 83.16      | 0.999942 |
| 2     | 49.37        | 48.56        | 82.97      | 84.36      | 0.999931 |
| 3     | 49.01        | 49.13        | 83.58      | 83.37      | 0.999998 |
| 4     | 48.59        | 49.37        | 84.29      | 82.97      | 0.999938 |
| 5     | 49.01        | 49.09        | 83.57      | 83.44      | 0.999999 |

Each trial delivered 4096 bytes for both flows according to fairness\_summary.csv. average throughput across the five trials was 48.90 B/s for Flow A and 49.08 B/s for Flow B. the largest throughput difference in any trial was only 0.81 B/s (about 1.66% of the pair average).

The result is highly repeatable under this controlled bottleneck: every trial has  $J > 0.99993$  and neither flow consistently wins. Flow A is slightly faster in Trial 2, while Flow B is slightly faster in Trials 1, 3, 4, and 5. However, the differences are very small, with the largest throughput difference being only 0.81 B/s, which provides no evidence of persistent starvation of either flow.



*Figure 19. Cumulative application bytes delivered by the two competing 4096-byte protocol flows during the shared-bottleneck fairness test. The traces remain nearly overlapped, showing similar application throughput for both flows.*

Taken together, the shares queue, repeated concurrent transfers, closely matches throughputs, and Jain index provide direct evidence that the protocol shares this bottleneck fairly under the testes condition. This is a controlled experimental claim rather than a claim that the protocol is fair on every possible network.

### 7.4 TCP Comparison (Experiment 2)

The shared-bottleneck trial in 7.3 measured our protocol against another instance of the same protocol. However, to measure whether it can coexist with genuinely different traffic, we decided to run it alongside a

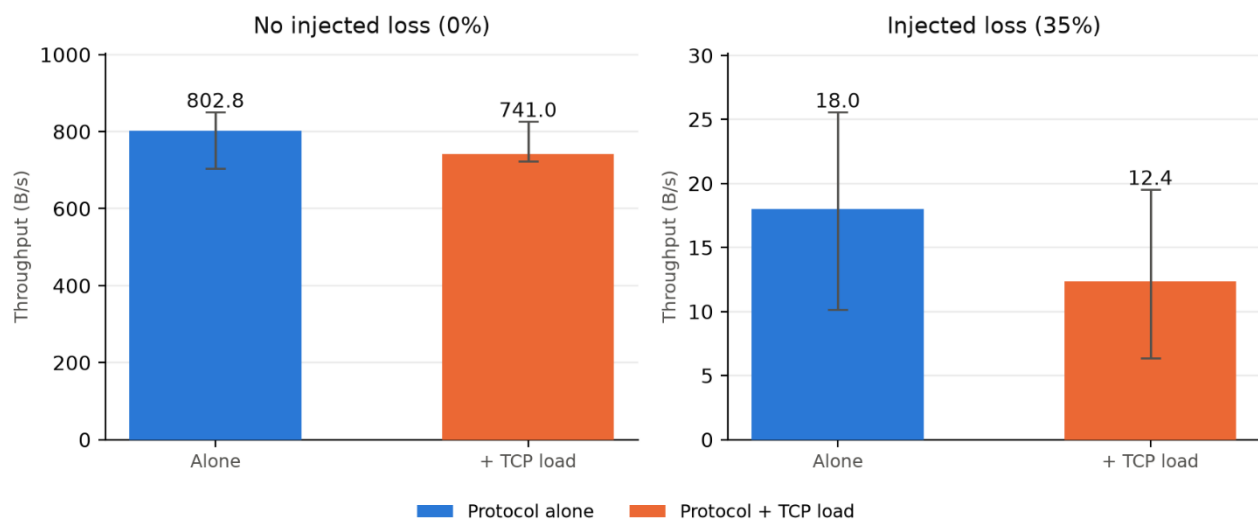
real TCP transfer. In this case, the competing load is a simple TCP socket pair on loopback, which transmits 64KB blocks continuously for the duration of the trial. The protocol was run under each condition, and the obtained transfer time was compared with a no-load baseline.

| Element                     | Configuration                                             |
|-----------------------------|-----------------------------------------------------------|
| Competing Traffic           | TCP socket pair on 127.0.0.1:9500, two concurrent senders |
| Block Size                  | 64KB per sendall()                                        |
| Our Protocol                | Ports 9001-9000                                           |
| Loss/Corruption rate tested | 0.0 / 0.35                                                |
| Trials                      | 5 per condition, 20 transfers in total                    |
| Payload                     | 289 B per transfer                                        |
| Competing Traffic           | TCP socket pair on 127.0.0.1:9500, two concurrent senders |

Both 0% and 35% loss/corruption rates were measured. At 0%, the transfer time doesn't vary over many trials, so any slowdown due to the competing traffic is visible. At 35%, the retransmission behavior instead produces greater run-to-run variation, which obscures the effect of this magnitude.

| Loss/Corruption Rate | Condition     | Avg Duration | Throughput | Range           |
|----------------------|---------------|--------------|------------|-----------------|
| 0%                   | Alone         | 0.36s        | 802.8 B/s  | 0.34 - 0.41s    |
| 0%                   | With TCP Load | 0.39s        | 741.0 B/s  | 0.35 - 0.40s    |
| 35%                  | Alone         | 16.04s       | 18.0 B/s   | 11.31 - 28.48 s |
| 35%                  | With TCP Load | 23.40s       | 12.4 B/s   | 14.81 - 45.26s  |

Protocol throughput with and without competing TCP traffic



*Figure 20. Protocol throughput with and without competing TCP traffic, at 0% and 35% loss/corruption rates. Error bars show the observed min and max across 5 trials.*

All twenty transfers delivered output that was identical to the input, byte-by-byte. At a 0% L/C rate, the designed protocol surrendered 8% of its throughput to competing traffic, reducing throughput from 802.8 B/s to 741.0 B/s. In comparison, at a 35% L/C rate, the measured difference was larger, at 1.46x; however, the trial ranges overlap widely, so this conclusion isn't as strong.

The result that provides better insight is the TCP throughput. We can see that TCP transferred approximately 9,300 MB/s ( $\approx 9.3$  GB/s) over the same interface, while our protocol only transferred 800 B/s, a difference of approximately 7 orders of magnitude. We can conclude that the two protocols were never competing for bandwidth. The 8% slowdown at 0% L/C rate can be explained by CPU and scheduler contention since both processes share the same machine. This test also showcases that our protocol coexists with unrelated TCP traffic without the loss of any correctness at a small cost in transfer time that in this case can be attributed to host scheduling rather than to the network.

## 8. Conclusion

---

This project demonstrates that reliable, connection-oriented transport can be constructed above UDP using application-level state and recovery logic. The sender and receiver synchronize with a three-way handshake, exchange sequenced segments, validate a custom checksum, reject corrupt or out-of-order data, pipeline transmissions, respect an advertised receive window, adapt a congestion window, estimate retransmission timeout, recover after controlled loss/corruption, and close explicitly with FIN / FIN-ACK / ACK.

The evidence is consistent across code, packet capture, reconstructed files, congestion logs, and repeated experiments. The 289-byte baseline payload is reconstructed exactly with the same hash. The Wireshark capture shows the handshake, `rwnd=4 ACK`, repeated `seq= 3578` after corruption, and graceful teardown. In the five-level performance sweep, every condition completed correctly in 5/5 trials while throughput fell and retransmissions rose as impairment increased. The no-loss CWND control reaches the configured 160-byte cap, whereas the 35% stress traces repeatedly reset to one MSS. Finally, for fairness, five 4096-byte shared-bottleneck fairness trials produced mean Jain  $J=0.999962$ , showing repeatable near-equal sharing under the tested condition, as well as the TCP comparison.

## References

---

- [1] J. Postel, "User Datagram Protocol," RFC 768, Aug. 1980.
- [2] W. Eddy (Ed.), "Transmission Control Protocol (TCP)," RFC 9293, Aug. 2022.
- [3] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," RFC 5681, Sept. 2009.
- [4] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's Retransmission Timer," RFC 6298, June 2011.
- [5] R. Jain, D.-M. Chiu, and W. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," DEC Research Report TR-301, Sept. 1984.
- [6] Project repository, "371-mini-project," GitHub, accessed Aug. 10, 2026.